Technische Hochschule Ingolstadt

Fakultät für Elektrotechnik und Informatik

Zukunft in Bewegung

Fluent Interfaces

Dokumenthistorie

Autor: Prof. Dr. Bernd Hafenrichter

Version: 1.1

Letze Änderung: 16.03.2025

Motivation

Grundidee

- Wie kann man seinen Code sprechend gestalten?
- Kann man ein Programm so gestalten das es sich wie der Satz einer natürlichen Sprache liest?
- Wie kann man verhindern das Schnittstellen mit komplexen Methoden falsch verwendet werden?

Lösungansatz: Fluent Interface

- Das Fluent Interface ist ein objektorientiertes API-Design, das es uns ermöglicht, Methodenaufrufe auf lesbare und intuitive Weise miteinander zu verknüpfen.
- Die verketteten Methodenaufrufe (Methodchaining) lesen sich wie eine natürlich Sprache

Motivation

Der traditionelle Ansatz ein Objekt zu erzeugen:

```
private void makeNormal(Customer customer) {
    Order o1 = new Order();
    customer.addOrder(o1);
    OrderLine line1 = new OrderLine(6, Product.find("TAL"));
    o1.addLine(line1);
    OrderLine line2 = new OrderLine(5, Product.find("HPK"));
    o1.addLine(line2);
    OrderLine line3 = new OrderLine(3, Product.find("LGV"));
    o1.addLine(line3);
    line2.setSkippable(true);
    o1.setRush(true);
  }
```

https://www.martinfowler.com/bliki/FluentInterface.html

Motivation

Der selbe Code mit einem Fluentinterface:

```
private void makeFluent(Customer customer) {
    customer.newOrder()
        .with(6, "TAL")
        .with(5, "HPK").skippable()
        .with(3, "LGV")
        .priorityRush();
  }
```

https://www.martinfowler.com/bliki/FluentInterface.html

Fluent Interfaces ohne feste Reihenfolge

Idee: Die Reihenfolge der verketteten Methodenaufrufe spielt keine Rolle

- Die Konstruktion eines Objektes hat keine definierte Reihenfolge
- Operationen müssen nicht zwingend aufgerufen werden

```
public interface FluentInterface {

    FluentInterface withArgA( String param );

    FluentInterface withArgB( String param );

    // Diese Methode beendet die Konstruktion
    // eines Objektes
    Result build();
}
```

Methodchaining: Verkettete Methodenaufrufe. Es wird immer das Fluentinterface als Returnwert zurückgegeben. Dies erlaubt den verketteten Aufruf und das bilden von Sätzen

Fluent interfaces ohne feste Reihenfolge

Idee: Die Reihenfolge der verketteten Methodenaufrufe spielt keine Rolle

```
public class FluentBuilder implements FluentInterface {

        private Result result = new Result();

        public static FluentInterface create() {
                return new FluentBuilder();
        }
        private FluentInterface withArgA(String param) {
                result.setArgA( param );
                return this;
        }
        public FluentInterface withArgB(String param) {
                result.setArgB( param );
                return this;
        }
        public Result build() {
                return result;
        }
}
```

Eine Beispielimplementierung für die Initialisierung eines Objektes.

Das Objekt „result“ wird schrittweise durch die einzelnen, verketteten Methoden initialisiert.

mittels build wird das Ergebnisobjekt zurückgegeben

Fluent interfaces ohne feste Reihenfolge

Idee: Die Reihenfolge der verketteten Methodenaufrufe spielt keine Rolle

```
Result result = FluentBuilder().create()
                               .withArgA( "a" )
                               .withArgB( "b" )
                               .build();
```

```
Result result = FluentBuilder().create()
                               .withArgB( "b" )
                               .withArgA( "a" )
                               .build();
```

```
Result result = FluentBuilder().create()
                               .build();
```

Fluent interfaces mit fester Reihenfolge (Progressive Interfaces)

Die Methodenaufrufe müssen in einer festen Reihenfolge ausgeführt werden

- Die Konstruktion eines Objektes erfordert eine definierte Reihenfolge
- Alle notwendigen Operationen müssen aufgerufen werden

```
public interface FluentInterface {
    FluentStep1 withArgA( String param );
}
```

```
public interface FluentStep1 {
    FluentEnd withArgB( String param );
}
```

```
public interface FluentEnd {
    Result build();
}
```

Das Fluentinterface wird in verschiedene einzelnen Interfaces aufgespalten (=Mediatoren)

Diese Mediatoren legen eine definierte Reihenfolge beim Methodchaining sicher und enthalten nur die im Kontext erlaubten Methoden

Fluent interfaces mit fester Reihenfolge (Progressive Interfaces)

```
public class FluentBuilder implements FluentInterface,
FluentStep1, FluentEnd {

private Result result = new Result();

        private FluentBuilder() {
        }
        public static FluentInterface create() {
                return new FluentBuilder();
        }
        public FluentStep1 withArgA(String param) {
                result.setArgA( param );
                return this;
        }

        public FluentEnd withArgB(String param) {
                result.setArgB( param );
                return this;
        }

        public Result build() {
        return result;
        }
}
```

- Eine Beispielimplementierung für die Initialisierung eines Objektes.
- Der FluentBuilder bietet über die create()-Methode den Einstiegspunkt in das Fluent Interface
- Das Objekt „result“ wird schrittweise durch die einzelnen, verketteten Methoden initialisiert.
- Die Mediatoren definieren die Aufrufreihenfolge
- mittels build wird das Ergebnisobjekt zurückgegeben

Fluent interfaces mit fester Reihenfolge (Progressive Interfaces)

Es können nur die Methodenaufrufe in der vorgegebenen Reihenfolge aufgerufen werden

```
Result result = FluentBuilder().create()
                               .withArgA( "a" )
                               .withArgB( "b" )
                               .build();
```

Zusammenfassung

Vorteile

- Gut lesbarer Code
- Sehr gut erweiterbar
- Auf bestehende Api‘s anwendbar
- Autovervollständigung lässt nur gültige Sätze zu

Nachteile

- Gute Fluent interfaces sind schwer zu erstellen
- Code wird umfangreicher
- Methoden alleine sind nicht mehr sprechend.
- Die Realisierung einer Grammatik für Fluent Interface ist sehr aufwendig
- Das Netzwerk von Mediatoren wird schnell unübersichtlich.